

Problem Set 2
COMP301 Spring 2025
Week 2: 28.02.2025 - 02.03.2025

Instructions:

- Submit your answers to the Blackboard PS2 assignment until **March 2nd** Sunday, at 23.59.
- Please submit only **one single PDF file**, where all of your codes for each of the parts are included.
- Name your submission file as *id_username_ps2.pdf*
(Example: *00000_shamdan17_ps2.pdf*).

Question Answering Part. In this section, you will answer the question in a paragraph.

Problem 1: What is the difference between recursive and iterative procedures? Explain their definitions and advantages over each other.

Coding Part. In this section, you will focus on recursive and iterative procedures.

Problem 2a: Sum of cubes, recursive. Implement a *recursive* procedure for calculating the sum of cubes of positive integers from 1 to N . You can **not** use the mathematical formula $\sum_{k=1}^N k^3 = \left(\frac{N(N+1)}{2}\right)^2$ to do so.

Some test cases are given below:

```
(sum-of-cubes-recursive 1) ; returns: 1
(sum-of-cubes-recursive 3) ; returns: 36 = 1^3 + 2^3 + 3^3 = 1 + 8 + 27 = 36
(sum-of-cubes-recursive 5) ; returns: 225
```

Problem 2b: Sum of cubes, iterative. Implement the same procedure in Problem 2a *iteratively*. *Hint: You can use a helper function.*

```
(sum-of-cubes-iterative 1) ; returns: 1
(sum-of-cubes-iterative 3) ; returns: 36
(sum-of-cubes-iterative 5) ; returns: 225
```

Problem 3: GCD. The greatest common divisor (GCD) of two integers a and b is the largest positive integer that divides both a and b without leaving a remainder. Mathematically, GCD is defined as:

$$\text{gcd}(a, b) = \max\{d \mid d \text{ divides } a \text{ and } d \text{ divides } b\}$$

For the following GCD problems, you can assume arguments a and b are positive integers and $a \geq b$. Some test cases for GCD are given below.

```
(gcd 48 18) ; returns: 6
(gcd 99 1) ; returns: 1
(gcd 17 13) ; returns: 1
(gcd 100 25) ; returns: 25
(gcd 42 42) ; returns: 42
```

Problem 3a - Naive GCD:. Implement an *iterative* procedure that naively computes the GCD by testing all the positive integers up to b , and returning the maximum.

Hint: You can define a helper function with additional arguments to keep track of values such as the current maximum.

Problem 3b - Euclidean algorithm: There is an optimized version of the GCD named Euclidean algorithm. The algorithm is as follows:

- Let $r = a \bmod b$
- Let $a := b$
- Let $b := r$
- If b is not 0, go to step 1
- Return a .

Implement the Euclidean algorithm.

Problem 3c - Discussion: Is your implementation of the Euclidean algorithm recursive or iterative, why?

1. Recursive solutions are functions that call themselves. They include a base case, and a recursive case and are coded using wishful thinking. A recursive function creates a new stack frame for each call.

Iterative solutions, on the other hand, are usually implemented in loops. But in Scheme, instead of loops, we use tail recursive functions. The difference is, the compiler is smart enough not to create new stack frames, which results in better performance for iterative solutions.

2. A.

```
(define (cubesum n) (if (= n 1) 1 (+ (* n n) (cubesum (- n 1)))))
```

B.

```
(define (cubesum_iterative n) (cubesum_loop n 1))
```

```
(define (cubesum_loop n S) (if (= n 1) S (cubesum_loop (- n 1) (+ S (* n n)))))
```

3. A.

Plan: a b, choose smallest, check all integers until $i \cdot i > \text{integer}$

```
(define (gcd a b) (if (< a b) (gcdh a b 1) (gcdh b a 1)))
```

```
(define (gcdh a b i res) (cond
```

```
((> i a) res)
```

```
((and (= (remainder a i) 0) (= (remainder b i) 0)) (gcdh (/ a i) (/ b i) 2 (* i res)))
```

```
(else (gcdh a b (+ i 1) res))))
```

B. (define (gcde a b) (if (= (remainder a b) 0) b (gcde b (remainder a b))))

Euclidean algorithm is iterative because it does not need to keep track of anything in the stack frame, it just returns the final result.